第7章 第3节 Resources 概念

- 来源: https://class.imooc.com/lesson/5279#mid=104932
- 阶段: 阶段三: 【工具与协议】让AI连接世界
- 周: 第7周 MCP：模型上下文协议
- 模块: Resources概念与实战
- 课节: 图文 1-1 Resources 概念

1-1 Resources 概念

第7章 第3节 Resources 概念

> [!TIP] 阅读指南 上一节了解了 MCP 的三大核心能力：Resources、Tools 和 Prompts。本节深入探讨 Resources 的工作原理。

3.1 Resources 要解决什么问题

之前讨论过 RAG——LLM 本身的知识是有限的，涉及企业商业机密、个人隐私的数据，可以通过 RAG 接入来补充。MCP Server 从某种意义上讲，也是一种 LLM 知识缺陷的解决方案，类似 RAG，但有本质的不同。

> 顺带一提，Qoder 的设置里可以一键安装 MCP，不妨现在就配置几个体验一下，后续也会讨论如何在 Qoder 中使用官方提供的 MCP。

一个真实的例子

假设在开发一个 AI 应用，它接入了一个「项目 API 文档」MCP Server。这个项目有 4 个模块的 API 文档：

```
authentication.md  → 用户认证相关API（登录、注册、登出、Token刷新）
user_profile.md    → 用户资料API（获取信息、修改头像、更新昵称、密码修改）
payment.md         → 支付相关API（创建订单、查询状态、退款处理、支付回调）
notification.md    → 通知推送API（发送消息、订阅设置、历史记录、已读标记）
```

现在问 AI：**"如何实现用户登录功能？需要调用什么接口？" 在没有 Resources 机制之前，面临一个两难选择：

```
# 方式1: 把所有API文档内容塞进 Prompt
prompt = f"""
API文档:
{认证API文档}   # 12KB
{资料API文档}   # 10KB
{支付API文档}   # 15KB
{通知API文档}   # 8KB

用户问题: 如何实现用户登录?
"""
# 调用 LLM
response = llm.chat(prompt)

# 问题: 浪费了 45KB tokens,实际只需要「认证API文档」（12KB）

#  方式2: 用 Function Calling 提供搜索工具
tools = [{
    "name": "search_api_docs",
    "description": "搜索API文档内容",
    "parameters": {"query": {"type": "string"}}
}]
# 问题: 这种工具本质上只是"关键词搜索", LLM 必须自己想搜索词，命中质量不稳定
#       搜索结果通常是若干相关片段，而不是一份结构清晰、可以整体阅读和复用的接口文档
```

两种方式各有短板：

- 方式1：简单但浪费——45KB 全塞进去，实际只需要 12KB；
- 方式2：依赖搜索工具，但 LLM 不知道有哪些具体的文档模块，构造的查询很难精准。

> Function Calling 本身没问题，问题是缺少一种机制让 LLM 知道「有哪些数据源」——需要对数据源给出详细的说明，帮助 LLM 做决策。 MCP 为这个问题提供了一个专门的机制：Resources。它的核心思路是——在真正读取内容之前，先把每个数据源的"名字"和"简介"列出来，让 LLM 先在"资源级别"做选择，再去读取具体内容。

3.2 Resources 如何帮助 AI 找到相关文档

这是 Resources 最核心的问题，一步步来拆解。

关键机制

Resources 不是神奇地「知道」该读哪个文档，而是通过 description 让 LLM 理解每个资源的内容。

下面是 MCP Server 暴露 Resources 时的一个典型结构：

```
# ✓ MCP Server 暴露结构化的 Resources
resources = [
    {
        "uri": "file://docs/authentication.md",
        "name": "用户认证API文档",
        "description": "用户认证相关的API接口，包括：用户登录（账号密码/手机验证码）、用户注册、登出、Token刷新和续期机制"
    },
    {
        "uri": "file://docs/user_profile.md",
        "name": "用户资料API文档",
        "description": "用户资料管理API，包括：获取用户信息、修改头像、更新昵称、修改密码、账号设置等功能接口"
    },
    {
        "uri": "file://docs/payment.md",
        "name": "支付API文档",
        "description": "支付相关API，包括：创建支付订单、查询支付状态、退款处理、支付回调通知处理等接口"
    },
    {
        "uri": "file://docs/notification.md",
        "name": "通知推送API文档",
        "description": "消息通知API，包括：发送站内消息、推送通知、订阅设置、历史记录查询、消息已读标记等功能"
    }
]
```

LLM 只要看这些结构化的 description，就能大致知道有哪些候选资源可以选。

```
工作流程：

1. MCP Server 提供 Resources 列表(包含 description)
   ↓
2. AI 应用把列表传给 LLM
   ↓
3. LLM 阅读所有 description,理解每个资源是什么
   ↓
4. LLM 根据用户问题，判断哪个资源最相关
   ↓
5. AI 应用请求 Server 读取被选中的 Resource，Server 执行文件读取后返回内容
   ↓
6. LLM 基于内容生成回答
```

注意这个流程中一个容易误解的地方：MCP Server 并不是远程服务器。在这个例子里，它就是一个跑在本地的进程——四个 API 文档就躺在项目目录的 docs/ 文件夹里，MCP Server 只是知道它们在哪、能在需要的时候把它们读出来。

> [!NOTE] 进程是什么？ 把它理解成电脑上正在运行的一个程序就行。你打开的微信、浏览器，每个都是一个独立的进程。MCP Server 本质上也是这样一个程序——只不过它不弹窗口，只在后台等着 AI 的指令。

MCP 也支持另一种形态：Server 连接远程服务（比如麦当劳的 MCP 下单服务，数据在麦当劳那边）。但协议是完全一样的——不管数据在哪，LLM 看到的都是同一套「列清单 → 选资源 → 读内容」的流程。先吃透本地数据这种最简单的情况，后面的变体自然能触类旁通。

具体示例

```
用户问题："如何实现用户登录功能？需要调用什么接口？"

LLM 的思考过程：

1. 看到 4 个 Resources（注意：只有 name 和 description，没有实际内容）：
   - "用户认证API文档" - description 提到 "用户登录（账号密码/手机验证码）、注册、登出、
Token刷新"
   - "用户资料API文档" - description 说的是 "获取用户信息、修改头像、更新昵称、修改密码"
   - "支付API文档" - description 说的是 "创建支付订单、查询支付状态、退款处理"
   - "通知推送API文档" - description 说的是 "发送站内消息、推送通知、订阅设置"

2. LLM 分析：
   用户问的是"用户登录功能" → 最匹配的是 "用户认证API文档"

3. LLM 告诉 AI 应用：
   "请读取 file://docs/authentication.md 这个 Resource"

4. AI 应用把请求转给 MCP Server，由 Server 从本地 docs/ 目录加载 authentication.md
（12KB），返回给 AI 应用

5. AI 应用把文件内容传给 LLM
   （现在 LLM 才看到完整的认证API文档）

6. LLM 在这12KB的文档中找到相关接口，回复用户：
   "要实现用户登录功能，需要调用认证API中的登录接口：

   POST /api/auth/login
   请求参数：
   - username: 用户名/手机号
   - password: 密码（需前端加密）
   - login_type: 登录方式（'password' 或 'sms'）

   返回数据包含 access_token 和 refresh_token，
   access_token 用于后续API调用，有效期2小时。.."
```

重点：

- 第1步：LLM 只看到 4 个 description（几百字），不是完整内容

第1步

- 第4步：为什么 AI 应用不能自己读文件，必须通过 Server？这是 MCP 安全模型的核心设计——本章后续会详细展开。
- 第5步：LLM 才看到选中的那个 Resource 的完整内容（12KB）

第5步

- 节省：另外3个文档的内容没有被加载（节省了约33KB tokens）

节省

- 合理大小：单个文档12KB左右，在LLM的处理范围内

合理大小

- 语义匹配：“用户登录” 和 “用户认证API” 有明显的语义关联，LLM能准确选择

3.3 下一节预告

理解了 Resources 的概念和核心机制后，下一节将进入实战环节，通过两个具体示例来展示 Resources 的两种实现方式：固定写死和动态生成，还会深入讲解如何编写高质量的 description 来提升资源匹配精度。

3.4 ■ 学点英语

中文	English	音标	说明
按需加载	On-Demand Loading	/ɑːn dɪˈmænd ˈloʊdɪŋ/	LLM先浏览Resource描述，确认需要时才读取完整内容的机制
精确浏览	Exact Browsing	/ɪgˈzækt ˈbraʊzɪŋ/	像翻目录一样手动选择资源的精准获取模式
资源标识符	Resource URI	/ˈriːsɔːrs juːɑːrˈaɪ/	唯一标识MCP中某个资源的自定义URI地址
语义模糊搜索	Semantic Fuzzy Search	/sɪˈmæntɪk ˈfʌzi sɜːrtʃ/	RAG中通过向量化自动匹配语义相似内容的方式